

SuryaGrid AI

Execution Plan: Substation-Driven Forecasting and DSM Workflow

Bengaluru / Karnataka / India Solar-Grid Forecasting
and DSM Multi-Agent Platform

Technical Execution Plan and Workflow Document

Author: Lekhan HR / SuryaGrid Project

Date: 7 July 2026

Baseline commit: 5bed3cd (substation-driven workflow foundation)

Scope: Next development phase; planning only (no app code changed)

Repositories: DBIT-Banglore/SuryaGrid, lekhanpro/SuryaGrid

This is a planning document. Every **IMPLEMENTED** item is backed by a repository file, model card, test, or command output verified this session. Items needing data the repository does not contain are marked **BLOCKED** with the required official source. Nothing is fabricated; un-run or absent capabilities are marked **PLANNED**, **PENDING**, or **NOT VERIFIED**.

Contents

1	Executive Summary	2
	Executive Summary	2
2	Current Verified Baseline	2
3	Problem Statement	3
4	Target Architecture	3
5	SubstationContext Specification	4
6	Detailed Agent Workflow	5
7	API Execution Plan	5
8	Frontend Execution Plan	6
9	DSM Engine Data Usage	7
10	Generation Timeline and Trace Design	7
11	Implementation Timeline	7
12	File-Level Implementation Plan	8
13	Testing and Acceptance Criteria	9
14	Risks and Mitigations	9
15	Next Phase After This Work	10

1 Executive Summary

SuryaGrid AI is an architecturally complete, partially data-complete prototype. The FastAPI backend exposes **61** `/api/v1` operations and passes **118** tests (`ruff check clean`); the Next.js 14 frontend builds (16 static routes). Real-data models are trained and carry cards: solar irradiance $R^2 = 0.956$, cloud-risk $F_1 = 0.67$ / $AUC = 0.90$, DSM-breach $F_1 = 0.79$ (Open-Meteo Bengaluru), plus a Kaggle Bengaluru irradiance model $R^2 = 0.920$ and cloud model $F_1 = 0.847$. 344 substations are ingested from OpenStreetMap (**capacity 0%**, **voltage 41%**; capacity is never fabricated).

As of commit 5bed3cd the **substation-driven workflow foundation is implemented**: a `SubstationContext` object, a context service over the real parquet, an orchestrator agent, five endpoints, honest DSM blocking, `agent_trace` / `calculation_trace`, and a frontend `SubstationWorkflowPanel` wired into the Locations and DSM pages, covered by 15 tests. **The remaining gap** is dashboard-wide substation context: the main dashboard still runs from a hardcoded location preset list, so this phase makes the selected `substation_id` the single active context for *every* dashboard card (weather, solar, cloud, generation timeline, DSM, source/limitation, agent-trace), adds a shared/persisted selection, reconciles the database-backed `/substations` list with the parquet-backed `/substations/catalog`, and hardens source-trace and missing-data surfacing across the UI.

Why substation-driven context is needed. A dropdown that only changes display text is cosmetic. Grid behaviour (irradiance at the substation coordinates, cloud risk, generation, deviation risk) is location-specific: the selection must change the *actual agent input context*, and every value must remain traceable with missing fields explicit. **Expected outcome:** selecting a substation refreshes all panels with values computed for that substation's coordinates and provenance; DSM output lists enabled vs blocked calculations honestly (no rupee/capacity/load fabrication); responses always carry an agent trace and a calculation trace; tests and the frontend build pass.

2 Current Verified Baseline

Table 1 is derived from the repository, model cards, `report_evidence_matrix.md`, `substation_data_quality_report.md`, and the 5bed3cd verification. Nothing is asserted beyond these sources.

Table 1: Current verified baseline (Table 1).

Area	Current verified status	Evidence	Limitation / next action
Backend API	IMPLEMENTED 61 <code>/api/v1</code> ops; 118 tests pass	route introspection; <code>pytest</code> -q	keep parity as UI wires in
Substation workflow (backend)	IMPLEMENTED context + orchestrator + 5 endpoints (5bed3cd)	<code>schemas/</code> , <code>services/</code> , <code>agents/</code> , <code>api/</code> files	combined only; per-card UI PLANNED
Agents	IMPLEMENTED 16 platform agents + 7-step orchestrator	<code>agents/*</code> ; <code>agent_trace</code>	weather live-fetch degrades to clear-sky
ML models	IMPLEMENTED 3 Open-Meteo + 2 Kaggle prod; PARTIAL PV/load non-prod; RL untrained	<code>models/metadata/*</code> cards	PV/load REAL_INDIA (domain shift)
ML datasets	IMPLEMENTED parquet + manifests present	<code>data/ml/*.parquet</code>	large parquets gitignored (reproducible)
Data sources	PARTIAL Open-Meteo + OSM live; NASA cross-check only; Kaggle used	<code>data_sources/</code> , registries	NASA live + SLD-C/CEA PLANNED

Area	Current verified status	Evidence	Limitation / next action
Substation data	IMPLEMENTED 344 OSM rows committed; capacity 0%, voltage 41%	bengaluru_-substations_-cleaned.parquet	BLOCKED needs KPTCL/BESCOM capacity
DSM	PARTIAL framework + ML breach; no rupee values	predict_dsm; emits_-rupee_values=false	BLOCKED needs KERC/CERC tariff order
Frontend	IMPLEMENTED builds (16 routes); panel on Locations+DSM	npm run build; SubstationWorkflowPanel.hsx	dashboard uses hardcoded preset
Deployment	PARTIAL single-instance docker-compose+nginx live (HTTP)	evidence matrix A1-A6	PLANNED DB record_counts= 0; ECS code-only
Lint	IMPLEMENTED ruff check clean; PARTIAL format-check 5 pre-existing files	verified this session	optional format-only follow-up

Must not be claimed (tracked **BLOCKED):** substation capacity (MVA), metered Karnataka/feeder load, official parsed KERC/CERC rupee tariff rates, local Bengaluru PV generation truth, deployed AWS ECS stack.

3 Problem Statement

Substations exist in the system (344 real OSM rows via `/substations/catalog`) and the backing endpoints exist (5bed3cd). The gap is that the selected substation is not yet the active context across the whole product surface:

- The main dashboard (`frontend/app/dashboard/page.tsx`) drives its cards from a hardcoded `LOCATIONS` preset array and `predictSite("primary-site", ...)`, not from a substation.
- The substation dropdown currently lives only inside `SubstationWorkflowPanel` on the `Locations` and `DSM` pages; it does not update the separate weather / solar / cloud / timeline / DSM / source / agent-trace cards, and there is no shared selection.
- The database-backed `/substations` list and the parquet-backed `/substations/catalog` can disagree (hosted DB `record_counts= 0` returns an empty list, while the catalog returns 344).

Requirement: the dropdown must change the *actual agent input context*. The selected `substation_id` must flow to the backend, become a `SubstationContext`, reach every agent, the DSM engine, and the orchestrator, and refresh every relevant panel with source trace, agent trace, and explicit missing/blocked handling.

4 Target Architecture

Figure 1 shows the request flow from selection to dashboard; Figure 2 shows the backend architecture with implemented (**IMPLEMENTED**) versus planned (**PLANNED**) elements.

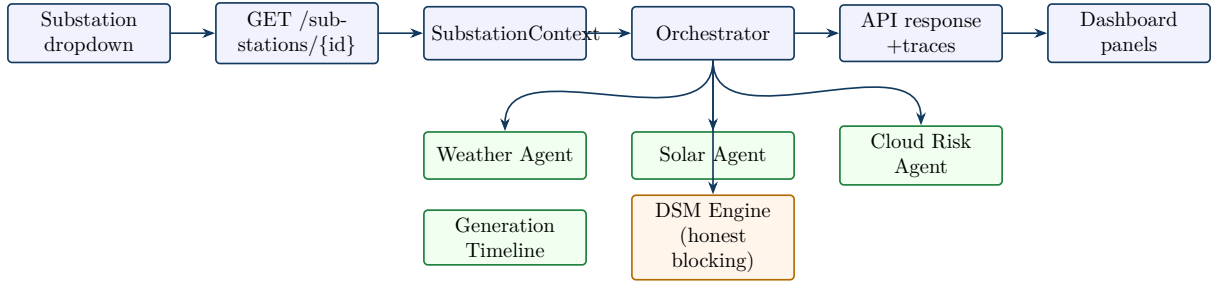


Figure 1: Selection → context → agents → dashboard (Figure 1). Blue = request path; green = agents; orange = DSM with honest blocking.

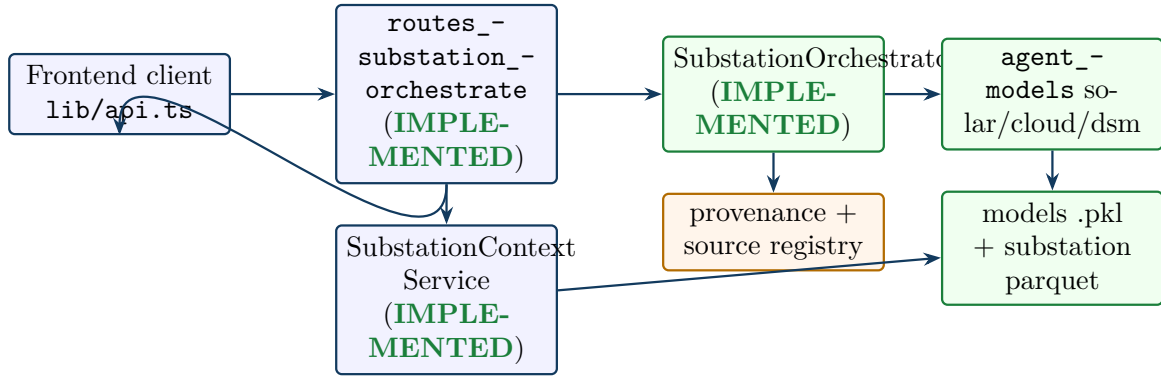


Figure 2: Backend architecture (Figure 2). All shown blocks are **IMPLEMENTED** at 5bed3cd; the frontend consumption of these across all dashboard cards is **PLANNED**.

5 SubstationContext Specification

Canonical object (backend/app/schemas/substation_context.py, **IMPLEMENTED**). Built by `SubstationContextService.get_context()`, which coerces `None` / `NaN` / `"nan"/"none"/"null"/""` to `null`.

```

substation_id, name, latitude, longitude          # identity/geography (always present)
operator          # 147/344 present, else null
voltage_kv        # 141/344 present, else null
capacity_mva      # 0/344 present -> ALWAYS null (never fabricated)
district          # 0/344 present -> ALWAYS null
state, source, source_url, reliability_score, missing_fields, data_geography, ingestion_time
display_label, distance_from_site_km, nearest_weather_location
weather_source    # "Open-Meteo @ (lat,lon) [REAL_COORDINATE_BASED]"
solar_source      # "solar_forecast_model.pkl [REAL_BENGALURU]"
tariff_region     # "Karnataka (KERC)"
source_status     # REAL_BENGALURU | REAL_KARNATAKA
capacity_status   # NOT_AVAILABLE (capacity null)
voltage_status    # source_status | NOT_AVAILABLE
load_data_status  # NOT_AVAILABLE
tariff_status     # NEEDS_OFFICIAL_SOURCE
limitation_notes  # human-readable honesty notes

```

Rules (enforced): missing `capacity_mva` stays `null` ($\rightarrow$ `capacity_status=NOT_AVAILABLE`); missing `voltage_kv` stays `null` ($\rightarrow$ `voltage_status=NOT_AVAILABLE`); `load` is `NOT_AVAILABLE`; `tariff` is `NEEDS_OFFICIAL_SOURCE`; no value is fabricated.

6 Detailed Agent Workflow

Implemented in `SubstationOrchestrator.run()` (**IMPLEMENTED**). Every step appends to `agent_trace`; every number records its formula + provenance in `calculation_trace`.

Table 2: Substation-driven agent workflow (Table 2).

#	Agent/Service	Input → processing	Output	Source trace / missing-data behaviour
1	Frontend selection	user click → substation_id	request w/ substation_id	disabled until catalog loads
2	Context Service	substation_id → load parquet, coerce, statuses	SubstationContext	REAL_BENGALURU/KARNATAKA; unknown id → 404; missing → null+status
3	Weather Agent	lat/lon → Open-Meteo @ coords	hourly weather + GHI	REAL_COORDINATE_BASED; net fail → pvlib clear-sky (labelled), never synthetic
4	Solar Agent	context+weather+model → predict_solar	irradiance (+est. PV if capacity)	REAL_BENGALURU; model missing → NOT_AVAILABLE
5	Cloud Risk Agent	weather features → predict_cloud	P(kt<0.5) drop risk	REAL_BENGALURU; model missing → NOT_AVAILABLE
6	Generation Timeline	weather+solar+cloud+substation_id	station rows (each carries substation_id)	ESTIMATED_FROM_IRRADIANCE; no capacity → MW null
7	DSM Engine	context+timeline+status → predict_dsm +gating	recommendation + blocked calcs	NEEDS_OFFICIAL_SOURCE; capacity/load/tariff missing → block+reason
8	Orchestrator	all outputs → assemble	response + agent_trace + calculation_trace	ESTIMATED_FROM_REAL; degraded steps flagged
9	Frontend render	response → update panels	refreshed dashboard	show warnings for missing/blocked

7 API Execution Plan

Base `/api/v1`; envelope `{success, message, data, timestamp}`.

Table 3: API endpoints: status and contract (Table 3).

Endpoint	Status	Request (data) → response	Consumer / acceptance
GET <code>/substations</code>	IMPLEMENTED	limit → DB rows (may be empty)	Locations table; list or explicit empty state
GET <code>/substations/catalog</code>	IMPLEMENTED	limit → 344 {id, display_label, coords, voltage, reliability}	Dropdown; loads 344
GET <code>/substations/{id}</code>	IMPLEMENTED	id → full SubstationContext (404 if unknown)	Detail card; missing null

Endpoint	Status	Request → response	Consumer / acceptance
POST /orchestrate/substation	IMPLEMENTED	{substation_id, site_capacity_mw?, scheduled_generation_mw?, forecast_horizon_hours, use_live_weather} → substation, trace, timeline, DSM, sources, limitations	Dashboard refresh; 7-step trace; is_synthetic=false
POST /dsm/forecast	IMPLEMENTED	same body → dsm_forecast (blocked calcs, no rupees)+trace	DSM panel; emits_rupee_values=false
GET /generation/timeline	IMPLEMENTED	?substation_id&site_capacity_mw&horizon&allow_estimated → timeline, summary, weather, trace	Timeline component; rows carry substation_id
/substations ↔ /catalog reconcile	PLANNED	delegate UI to catalog or seed DB from parquet	one authoritative list on host

No new endpoints are strictly required; the only **PLANNED** API work is reconciling the DB list with the parquet catalog so the dropdown is never empty on the hosted deployment.

8 Frontend Execution Plan

Verified components: ModelProvenancePanel, OfflineBanner, SubstationWorkflowPanel (new), cards/{MetricCard, PenaltyStatusCard}, charts/{DeviationBar, MiniTimeline}, svg/{SolarPanel3D, RiskGauge3D, EnergyFlow3D}. The dashboard currently uses a hardcoded LOCATIONS array.

Table 4: Frontend changes (Table 4).

Component / page	Current behaviour	Required change	Endpoint / acceptance
lib/api.ts	IMPLEMENTED catalog/context/orchestrate/dsm/timeline fns	reuse; shared fetch on selection	no mock in real mode
Substation dropdown	in panel on 2 pages	promote to shared selector (store/query param)	/substations/catalog; persists across pages
app/dashboard/page.html	hardcoded LOCATIONS + predictSite	add selector; drive cards from orchestrate	/orchestrate/substation; selecting refreshes
Detail / weather / solar / cloud cards	preset-driven / in panel	bind to orchestrate response	selected coords + provenance shown
Generation timeline (MiniTimeline)	getTimeline(...)	bind to /generation/timeline	rows carry substation_id
DSM panel	in panel (dsm page)	dedicated dashboard card	blocked calcs + no rupees
Source/limitation + agent-trace panels	in panel	dashboard-wide panels	provenance + 7-step trace always visible

Guardrail: no redesign; reuse glass-card/input-field/ btn-primary; never render mock data when the backend is reachable; keep the OfflineBanner behaviour.

9 DSM Engine Data Usage

DSM step: `SubstationOrchestrator._build_dsm_forecast + agent_models.predict_dsm` (**IMPLEMENTED**). **Input:** `selected_substation` (context), `generation_timeline`, `solar_forecast`, `cloud_risk`, `load_data_status`, `tariff_source_status`, `capacity_status`, `voltage_status`, `source_trace`. **Output:** `recommendation`, `action`, `risk/deviation_band`, `confidence`, `fields used` (`context_inputs_used`), `fields missing`, `calculations enabled`, `calculations blocked`, `reason`, `limitation notes`.

Table 5: Blocked-calculation rules (enforced) (Table 5).

Condition	Blocked calculation	Status flag
capacity_mva missing (always, OSM)	substation_loading_percent	capacity_status=NOT_AVAILABLE
load data missing	load_following_optimisation	load_data_status=NOT_AVAILABLE
official tariff missing	dsm_rupee_charge (settlement/savings)	tariff_status=NEEDS_OFFICIAL_SOURCE; emits_rupee_values=false
PV generation unavailable	mark generation ESTIMATED	actual_generation_available=false
voltage_kv missing	voltage_band_optimisation	voltage_status=NOT_AVAILABLE

10 Generation Timeline and Trace Design

Timeline rows (**IMPLEMENTED**; **PLANNED** items marked): `timestamp`, `substation_id`, `substation_label` (**PLANNED** explicit `substation_name`), `weather features` (`clearsky_ghi_wm2`, `observed_ghi_wm2`; **PLANNED** per-row `temp/cloud`), `forecast_ghi_wm2`, `estimated_generation_mw` (if `capacity`; else `null`), `cloud_drop_risk`, **PLANNED** `confidence_components`, `data_sources`, `limitations`. Rule: no actual generation $\rightarrow$ `generation_type=ESTIMATED_FROM_IRRADIANCE`, `actual_generation_available=false`.

Traces. `agent_trace[]` carries `agent`, `status` (`ok/degraded/blocked/not_available`), `action`, `source_label`, `detail`. `calculation_trace{}` keys (**IMPLEMENTED**): `clearsky_ghi_wm2`, `forecast_ghi_wm2`, `cloud_drop_risk`, `estimated_generation_mw`, `scheduled_ghi_estimate_wm2`, `deviation_percent` (each with `formula`, `inputs`, `source_label`, `model_file`). **PLANNED** enrichment to fully match the requested schema:

```
{ "selected_substation_fields_used": [...], "selected_substation_fields_missing": [...],
  "weather_features_used": [...], "solar_features_used": [...],
  "grid_features_used": [], "tariff_features_used": [], "load_features_used": [],
  "model_files_used": ["solar_forecast_model.pkl", "cloud_risk_classifier.pkl", "dsm_classifier.pkl"],
  "formulas_or_rules_used": ["Ineichen clearsky", "PV proxy PR=0.80", "deviation %"],
  "calculations_skipped": ["substation_loading_percent", "dsm_rupee_charge"],
  "skip_reasons": {"substation_loading_percent": "capacity_mva NOT_AVAILABLE"} }
```

11 Implementation Timeline

Durations are working-day estimates. Because the backend foundation shipped in 5bed3cd, the critical path is frontend wiring, trace enrichment, and tests. Estimated remaining effort: **5–8 working days** (frontend-dominant).

Table 6: Phase plan / Gantt-style summary (Table 6).

Ph	Focus	Status	Dur	Backend	Frontend	Deliverable
A	Audit + endpoint map	VERIFIED done	0.5d	audit	audit	this plan
B	Context schema/service	IMPLEMENTED		context+coerce	—	schema+service
C	Backend API integration	IMPLEMENTED		5 endpoints	—	endpoints
D	Agent integration	IMPLEMENTED		weather/solar/cloud	—	7-step trace
E	DSM integration	IMPLEMENTED		honest blocking	—	DSM forecast
F	Generation time-line	IMPLEMENTED		rows w/ id	—	timeline
G	Dashboard wiring	PLANNED	2–3d	parity only	selector+cards	wired dash-board
H	Source-trace + missing-data UI	PLANNED	1–2d	enrich trace fields	trace/source panels	full provenance UI
I	Tests + validation	PARTIAL	1–2d	extend tests	build+UI tests	green tests
J	Docs + deploy verify	PARTIAL	1d	host check	—	deploy note

Dependencies: G depends on B–F (done); H depends on G; I depends on G,H; J depends on I.

12 File-Level Implementation Plan

Actual repository paths (backend `backend/app/api/`, frontend `frontend/app/` App Router).

Table 7: File-level plan (Table 7).

File path	Action	Reason / acceptance
<code>backend/app/schemas/substation_context.py</code>	IMPLEMENTED	context object; fields+statuses present exists
<code>backend/app/services/substation_context_service.py</code>	IMPLEMENTED	load parquet, coerce, catalog; 344 rows exists
<code>backend/app/agents/substation_orchestrator.py</code>	IMPLEMENTED / PLANNED	7-step; add explicit trace arrays enrich
<code>backend/app/api/routes_substation_orchestrate.py</code>	IMPLEMENTED	5 endpoints registered exists
<code>backend/app/api/routes_locations.py</code>	PLANNED	up- reconcile /substations with catalog date
<code>backend/data/ml/bengaluru_substations_cleaned.parquet</code>	IMPLEMENTED	real source (24 KB), 344 rows committed
<code>frontend/lib/api.ts</code>	IMPLEMENTED	typed client fns; no mock exists
<code>frontend/lib/ (new substation store/context)</code>	PLANNED	create new shared selection persists
<code>frontend/app/dashboard/page.tsx</code>	PLANNED	up- add selector; drive cards from orchestrate
<code>frontend/app/locations/page.tsx, .../dsm/page.tsx</code>	IMPLEMENTED	selection entry points wired
<code>frontend/components/SubstationWorkflowPanel.tsx</code>	IMPLEMENTED / PLANNED	optionally split into per-card components refactor
<code>frontend/components/ (new cards)</code>	PLANNED	create/update weather, solar, cloud, DSM, source, trace cards bind to orchestrate response

File path	Action	Reason / acceptance
docs/ workflow + DSM-input-trace	IMPLEMENTED	both docs updated to reflect 5bed3cd updated

13 Testing and Acceptance Criteria

Backend (backend/tests/test_substation_workflow.py, **IMPLEMENTED** 15 tests): valid substation returns context; missing capacity stays null; orchestrator uses selected lat/lon; timeline rows contain selected substation_id; DSM receives selected substation; DSM blocks capacity when capacity missing; blocks load optimisation when load missing; blocks rupee calc when tariff missing; agent_trace present; calculation_trace present; APP_DATA_MODE=real blocks synthetic. **PLANNED**: enriched calculation_trace arrays. **Frontend/build** (**PLANNED**): dropdown loads from API; selection refreshes cards; DSM panel shows missing-data warnings; timeline updates; npm run build passes (**IMPLEMENTED** baseline).

```
cd backend && python -m pytest tests/ -q
cd frontend && npm run build
```

Acceptance checklist (phase gate). Met at 5bed3cd: dropdown loads 344; substation_id reaches backend and appears in orchestrator response; timeline uses selected coordinates; DSM includes selected substation; agent_trace + calculation_trace present; missing fields + blocked calculations explicit; no fabricated capacity/voltage/load/tariff; 118 tests pass; docs updated. **Open item (PLANNED)**: frontend updates *all* dashboard sections on selection (Phase G/H), and /substations/catalog verified non-empty on the hosted deployment (Phase J).

14 Risks and Mitigations

Table 8: Risks and mitigations (Table 8).

Risk	Impact	Mitigation	Status
OSM capacity missing (0%)	no loading/capacity DSM	keep null + block; require official source	BLOCKED KPT-CL/BESCOM
Voltage partial (59% missing)	no voltage logic for some	per-substation status; block when null	PARTIAL handled
No real substation load	no real load optimisation	load_data_status=NOT_AVAILABLE; block	BLOCKED SLDC
No local PV truth	generation is estimate	ESTIMATED_FROM_IRRADIANCE; not measured	PARTIAL handled
Tariff parsing pending	no rupee settlement	framework-only; emits_rupee_values=false	BLOCKED KERC/CERC
FE/BE API mismatch	broken cards	typed client; build gate; contract tests	PLANNED G/I
DB /substations vs catalog mismatch	empty dropdown on host	delegate to catalog or seed DB	PLANNED G/J
Synthetic fallback leakage	fabricated data in real mode	APP_DATA_MODE=real guard; clear-sky (not synthetic)	VERIFIED enforced+tested
AWS deployment mismatch	ECS assumed vs compose live	treat ECS code-only until verified	PARTIAL documented

15 Next Phase After This Work

1. **BLOCKED** Official substation enrichment: KPTCL/BESCOM/CEA capacity (MVA) + transformer ratings → unblock `substation_loading_percent`.
2. **BLOCKED** Karnataka load integration: SLDC / Grid India feeder/substation load → unblock real load-following optimisation.
3. **BLOCKED** Official tariff parser: KERC/BESCOM DSM tariff order → unblock rupee settlement/savings.
4. **PARTIAL** AWS backend/API hardening: verify ECS/Terraform (currently code-only); HTTPS/cert.
5. **BLOCKED** Real PV generation ingestion: Bengaluru/Karnataka metered PV → production PV model.
6. **PLANNED** RL environment: only after real load + tariff + PV exist (`rl_policy` remains untrained: `INSUFFICIENT_REAL_ENVIRONMENT_DATA`).

Immediate next implementation step (Phase G). Add a shared substation selector to `frontend/app/dashboard/page.tsx` that calls `POST /api/v1/orchestrate/substation` and binds the weather, solar, cloud, generation-timeline, DSM, source/limitation, and agent-trace cards to the response, reusing the existing `lib/api.ts` client functions and `glass-card` styling, with no mock data when the backend is reachable.

*This plan modifies no application code. All **IMPLEMENTED** claims correspond to files/tests present at commit `5bed3cd`; all **BLOCKED** items require the named official data source and are not asserted as available.*